TASK 3 — SECURE CODING REVIEW

Objective

Review a small Python application for security weaknesses using manual inspection, document the findings, and recommend remediation steps. The CodeAlpha task specifies selecting a programming language and application, identifying vulnerabilities, and suggesting secure-coding practices.

Language and Application

Language: Python
Application: Simple user login function
Review method: Manual secure-coding inspection

Sample Code Under Review

```
username = input("Username: ")
password = input("Password: ")

query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'"
cursor.execute(query)

if cursor.fetchone():
print("Login successful")
else:
print("Invalid login")
```

Finding 1 — SQL Injection Risk

The SQL statement is constructed by directly concatenating user-controlled input. A malicious input can change the intended SQL query. This is a high-impact design weakness because it can affect authentication and database access.

Remediation

Use parameterized queries / prepared statements instead of string concatenation. Validate inputs according to application requirements and use a database account with only the permissions the application needs.

Finding 2 — Plain-Text Password Handling

The example compares a password directly in a database query. A secure application should not store user passwords as plain text. Passwords should be protected using a suitable password-hashing mechanism and verified through the application's authentication framework.

Finding 3 — Excessive Error Information

Although the sample does not expose a detailed database error, production applications should avoid returning internal database or stack-trace information to users. Detailed diagnostics should be recorded securely in logs while users receive safe, generic messages.

Secure Version — Example

```
username = input("Username: ")
password = input("Password: ")

query = "SELECT password_hash FROM users WHERE username = ?"
cursor.execute(query, (username,))

row = cursor.fetchone()

if row and verify_password(password, row[0]):
print("Login successful")
else:
print("Invalid username or password")
```

Additional Secure Coding Recommendations

• Apply least-privilege permissions to database accounts.
• Keep dependencies and frameworks updated.
• Protect secrets and never hard-code credentials in source code.
• Validate input according to expected type, length, format, and business rules.
• Implement appropriate authentication and authorization checks on the server side.
• Log security-relevant events without exposing sensitive information.
• Review code regularly and test security controls after remediation.

Summary of Findings

Finding	Risk	Recommended Remediation
SQL injection	Unsafe query construction	Parameterized queries / prepared statements
Password handling	Sensitive credential exposure	Secure password hashing and verification
Error handling	Possible information leakage	Generic user errors + protected logs

Conclusion

The review identified insecure SQL query construction, unsafe password handling, and the need for controlled error handling. Applying parameterized queries, secure password verification, least privilege, input validation, protected logging, and regular dependency updates improves the application's security posture.